

DataKern Data Engineering — Assignment 4

Spark Fundamentals: Distributed Processing & the Spark UI

Continuation of Assignment 3: Same Olist business data. A different processing engine.

Your role: Data Engineer, E-commerce Data Team **Deliverable:** One Jupyter notebook — notebooks/assignment_4_spark.ipynb

1. Your next assignment

Your team lead reviewed the PostgreSQL pipeline you built in Assignment 3. You modelled the data, validated it, and produced Finance and Operations outputs.

Then your team lead asks:

"Good work. Now imagine the dataset is one hundred times bigger. Would the same approach still be the right choice?"

A single-machine database can be an excellent choice for many workloads. But as data volume and processing requirements grow, you may need to distribute work across multiple machines.

Your team uses **Apache Spark**.

Your task is to solve the same Finance and Operations questions using PySpark, while learning how Spark plans and executes work. The objective is not to make this small Olist dataset run faster. The objective is to understand the execution model well enough to reason about a much larger workload.

Learning rule: Do not use AI code-generation or coding-agent tools to produce your solution. This includes ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar tools. You may use official documentation and search for errors, but you must write, test, and explain your own solution.

2. What you will learn

By the end, you should be able to:

- Explain lazy evaluation, transformations, actions, logical plans and physical plans.
- Explain partitions, tasks, stages, DAGs and shuffle boundaries.
- Distinguish narrow and wide transformations.
- Use core PySpark DataFrame operations.
- Understand repartitioning, coalescing, caching and broadcast joins.
- Read basic execution information in the Spark UI.
- Build and validate Finance and Operations outputs using Spark.
- Explain when Spark may be appropriate — and when a database may be simpler.

3. What you will build

One notebook, organised into five parts:

Part 1 – Environment, SparkSession and data ingestion
Part 2 – Transformations and the execution model
Part 3 – Actions, caching, partitions and Parquet
Part 4 – Finance and Operations business outputs
Part 5 – Spark UI observations and engineering reflection

No databases, cloud services, Airflow or cluster deployment. Use the Olist CSV files from your existing repository and run Spark locally.

4. Before you code: build the mental model

4.1 Transformations and actions

A **transformation** describes a new DataFrame. Spark generally records the operation in a plan rather than immediately processing all its rows.

An **action** asks Spark to produce a result. It can trigger execution of the transformations required to produce that result.

Operation	Classification	What to remember
<code>select()</code> , <code>filter()</code> , <code>withColumn()</code>	Transformation	Builds a plan
<code>join()</code> , <code>groupBy()</code>	Transformation	Builds a plan; execution may require a shuffle
<code>count()</code> , <code>show()</code> , <code>collect()</code>	Action	Triggers the required computation
<code>write...</code>	Action	Executes and writes output
<code>printSchema()</code>	Schema inspection	Displays schema; does not scan all rows
<code>explain()</code>	Plan inspection	Displays plans; does not execute the DataFrame query

This is the central idea behind **lazy evaluation**. Do not interpret it as “Spark never does work until an action”: operations such as schema inference may need to inspect input data.

4.2 Narrow and wide transformations

A partition is a logical chunk of a DataFrame that Spark can process as a unit of parallel work.

- **Narrow dependency:** A child partition can be computed from a limited set of parent partition(s). Operations such as `filter()` and `select()` are typical examples.
- **Wide dependency:** A child partition may need data from many parent partitions. Operations such as `groupBy()` and `repartition()` commonly require data redistribution, called a **shuffle**. Joins often shuffle, depending on the join strategy and data distribution.

A shuffle is a boundary where Spark redistributes data between partitions. In a real cluster, this can involve network transfer, disk I/O and serialization.

4.3 Jobs, stages and tasks

- An **action** can create one or more jobs.
- A **job** is divided into **stages** at shuffle boundaries and other execution boundaries.
- A **task** is the unit of work sent to an executor for a partition in a stage.

Do not assume one action always means one job, one stage or one task. A stage's task count is generally related to the number of partitions it processes. Different stages in the same job can have different task counts.

4.4 The execution journey

```

Your DataFrame code
    |
    v
Logical plan
    |
    v
Optimised logical plan
    |
    v
Physical plan
    |
    v
DAG / stages separated by shuffle boundaries
    |
    v
Tasks process partitions
    |
    v
Action returns a result or writes output
  
```

Pause and write your initial answers in a Markdown cell:

1. If you call `filter()` twice and then `count()`, how many times do you expect the source to be read? What might change that?
2. What problem does a DAG help Spark represent?
3. What is the difference between a stage and a task?
4. Why can a `filter()` usually work without moving rows between partitions, while a `groupBy()` often cannot?

Return to these answers at the end and improve them.

5. Environment and project setup

Continue using your Olist repository. Add:

```

your-olist-repository/
|-- data/
|   |-- raw/
|   |-- <Olist CSV files>
|-- notebooks/
|   |-- assignment_4_spark.ipynb
  
```

```
|-- output/                # generated locally; do not commit
|-- .gitignore
|-- requirements.txt
```

5.1 Check your environment

PySpark requires a compatible Java runtime. Use the Java and Spark compatibility guidance for the PySpark version you install. Before starting the notebook, verify:

```
python --version
java -version
```

Install PySpark in your active virtual environment:

```
pip install "pyspark>=3.4,<5"
```

Record your Python, Java and PySpark versions in the notebook. If Java is missing or incompatible, resolve that before continuing. Do not copy random environment variables from online answers without understanding what they do.

5.2 Start a SparkSession

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("DataKern-Olist-Spark")
    .master("local[*]")
    .config("spark.ui.port", "4040")
    .getOrCreate()
)
```

`local[*]` runs Spark locally and uses the available logical CPU cores. It is suitable for learning, but it does not reproduce all the behaviour of a multi-machine cluster.

Open `http://localhost:4040` while the Spark application is running. If port 4040 is already in use, Spark may select another available port; check the notebook output for the actual Spark UI URL.

Pause and observe:

- What does `master("local[*]")` mean?
- What does `[*]` represent?
- Which Spark UI tabs are visible?
- Why might the Jobs page be empty at this point?
- Take a screenshot of the UI before running a data action.

6. Part 1 — Read the Olist data

At minimum, load:

```
olist_orders_dataset.csv
olist_customers_dataset.csv
olist_order_items_dataset.csv
olist_products_dataset.csv
olist_product_category_name_translation.csv
```

Use paths relative to your repository rather than machine-specific absolute paths.

6.1 Compare schema inference with an explicit schema

First load the orders file using inference:

```
orders_inferred = spark.read.csv(
    "data/raw/olist_orders_dataset.csv",
    header=True,
    inferSchema=True
)
```

Then load it again using an explicit schema. Define suitable types for the columns you need. At minimum, identify order and customer IDs as strings and date/time fields as timestamps or strings that you deliberately parse later. Use the appropriate Spark SQL types.

```
from pyspark.sql.types import (
    StructType, StructField, StringType, TimestampType
)

orders_schema = StructType([
    # Define the fields and data types you need.
])

orders = spark.read.csv(
    "data/raw/olist_orders_dataset.csv",
    header=True,
```

```
        schema=orders_schema
    )
```

Complete the schema yourself; do not leave the placeholder in your submitted notebook.

Investigate:

1. Compare `printSchema()` for both DataFrames.
2. Observe the Spark UI before and after loading with `inferSchema=True`.
3. Does schema inference need to inspect the file? What execution or scan activity do you observe in your environment?
4. Why can inference be expensive for large inputs?
5. What benefits does an explicit schema provide?

Important: `printSchema()` displays schema information; it is not a full-data action. Schema inference, by contrast, can scan input data. Do not assume every CSV read is equally lazy.

Load the remaining files. Use explicit schemas where practical, or document why you chose another approach. Inspect each schema.

6.2 First action: count

Run:

```
orders.count()
```

Record:

- The returned row count.
- Jobs created.
- Stages and tasks shown in the UI.
- What the task count suggests about the input partitions.

Do not hardcode expected task counts. They depend on file splitting, Spark settings and the execution plan.

7. Part 2 — Transformations: build and inspect the plan

For each exercise:

1. Write down your prediction before running the code.
2. Create the DataFrame.
3. Inspect its schema or plan where relevant.
4. Trigger the specified action.
5. Inspect the Spark UI.
6. Record what happened and explain why.

7.1 `select()` — choose columns

```
orders_slim = orders.select(
    "order_id",
    "customer_id",
    "order_status",
    "order_purchase_timestamp"
)
```

Predict whether this statement alone creates a Spark job. Then trigger a suitable action and observe.

Explain the difference between `select()` and `drop()`.

7.2 `filter()` — narrow the data

```
delivered_orders = orders_slim.filter(
    orders_slim.order_status == "delivered"
)
```

Before calling `count()`, estimate the number of delivered orders. Then run the count.

Explain:

- Why is `filter()` a narrow transformation in the usual case?
- How many tasks did the action use?

- What does that task count tell you — and what does it not tell you?

7.3 withColumn() — derive columns

Parse the purchase timestamp and derive year and month:

```
from pyspark.sql.functions import col, year, month, to_timestamp

orders_with_month = (
    delivered_orders
    .withColumn(
        "purchase_ts",
        to_timestamp(col("order_purchase_timestamp"))
    )
    .withColumn("purchase_year", year(col("purchase_ts")))
    .withColumn("purchase_month", month(col("purchase_ts")))
)
```

Predict how many columns are added. Does chaining these transformations necessarily mean separate Spark jobs? Explain what the plan shows.

7.4 join() — combine data

```
orders_with_items = orders_with_month.join(
    order_items,
    on="order_id",
    how="inner"
)
```

Predict whether the row count will increase, decrease or stay the same. Explain your prediction using the grain of each dataset.

Trigger count() and inspect the SQL tab / query plan.

Investigate:

- Which join strategy did Spark choose?
- What is a Sort Merge Join?
- What is a Broadcast Hash Join?
- Why might Spark choose one strategy over another?

Optional controlled experiment — broadcast: Use a genuinely small lookup DataFrame and try broadcast():

```
from pyspark.sql.functions import broadcast

# Join a larger DataFrame to a small lookup DataFrame.
# Compare the physical plan with and without broadcast().
```

Do not broadcast a large DataFrame just to force a different plan. Explain why broadcasting a large dataset can be harmful.

7.5 groupBy() and agg() — aggregate

```
from pyspark.sql.functions import (
    countDistinct, sum as spark_sum, round as spark_round
)

monthly_summary = (
    orders_with_items
    .groupBy("purchase_year", "purchase_month")
    .agg(
        countDistinct("order_id").alias("distinct_orders"),
        spark_round(spark_sum("price"), 2).alias("total_item_sales")
    )
    .orderBy("purchase_year", "purchase_month")
)

monthly_summary.show(24)
```

Investigate:

- How many jobs, stages and tasks appeared?
- Can you identify an Exchange / shuffle in the plan?
- Why does grouping commonly require records with the same key to meet in the same partition?
- What data is redistributed?
- Why can excessive shuffle slow down a distributed pipeline?

7.6 Connect transformations to stages

Use the examples you have already run to classify:

Operation	Typical dependency	Typical shuffle?
<code>select()</code>	Narrow	No
<code>filter()</code>	Narrow	No
<code>withColumn()</code>	Narrow	No
<code>groupBy()</code>	Wide	Yes
<code>repartition()</code>	Wide	Yes
<code>join()</code>	Depends on strategy	Often

These are typical behaviours, not unconditional guarantees. The physical plan and runtime configuration matter.

In your own words, explain how a shuffle boundary can divide a job into stages.

7.7 `dropDuplicates()` — validate data grain

Check uniqueness at the order-item grain:

```
total_rows = order_items.count()

distinct_rows = (
    order_items
    .dropDuplicates(["order_id", "order_item_id"])
    .count()
)

print("Total rows:", total_rows)
print("Distinct rows:", distinct_rows)
print("Duplicate keys:", total_rows - distinct_rows)
```

Then create:

```
order_items_clean = order_items.dropDuplicates(
    ["order_id", "order_item_id"]
)
```

Answer:

- What is the difference between `dropDuplicates()` with and without a subset?
- What columns are compared when no subset is provided?
- What happens if two rows share the same key but have different values in other columns?
- Why is “keep any row” not always a safe business rule?

Controlled duplicate experiment: If the source contains no duplicate keys, create a tiny separate test DataFrame with a deliberately duplicated row. Demonstrate deduplication on that test data. Do not alter the original Olist source files.

In a production pipeline, define the expected grain and a rule for resolving conflicting duplicates before deduplicating.

8. Part 3 — Actions, partitions, cache and output

8.1 Understand common actions

Use the small `monthly_summary` DataFrame for actions that return rows:

Action	What it does	Caution
<code>count()</code>	Returns row count	Scans the required input
<code>show(n)</code>	Displays up to n rows	Useful for inspection
<code>first()</code>	Returns the first row	Result order is not guaranteed without ordering
<code>take(n)</code>	Returns up to n rows	Keep n small
<code>collect()</code>	Returns all rows to the driver	Can exhaust driver memory
<code>write...</code>	Writes data to storage	Executes the write

For each action, record the job, stages, tasks and approximate duration. Repeated actions may create repeated jobs unless useful results are cached or otherwise reused.

Do not run `collect()` on the full `orders_with_items` DataFrame. Explain why collecting a large DataFrame to the driver is dangerous. Use `take(5)` or `show(5)` for small samples.

8.2 Cache only when reuse justifies it

Caching is useful when a DataFrame is expensive to compute and will be reused. It is not automatically beneficial: caching consumes resources and may add overhead.

For a focused experiment, cache the Finance base DataFrame you will actually reuse:

```

finance_base = (
    orders.join(order_items, on="order_id", how="inner")
    .filter(col("order_status") == "delivered")
)

finance_base.cache()
finance_base.count() # materialise the cache

```

Use this same `finance_base` in your Finance analysis rather than rebuilding the same join from the raw DataFrames.

Inspect the Storage tab:

- Is the cached DataFrame listed?
- How much is cached in memory or disk?
- What happens if the cache cannot fit entirely in memory?

After the Finance work is complete:

```
finance_base.unpersist()
```

Important: Do not reuse this delivered-orders DataFrame for Operations. The Operations analysis needs to consider orders that have not yet been delivered.

For a separate optional experiment, compare recomputation with caching on a DataFrame that is genuinely reused. Explain whether caching helped in your local run; do not claim that a single small local benchmark proves a general performance improvement.

8.3 repartition() and coalesce()

Partitions influence parallelism and the amount of work available to tasks.

```

print("Original:", orders_with_items.rdd.getNumPartitions())

orders_repartitioned = orders_with_items.repartition(8)
print("After repartition:", orders_repartitioned.rdd.getNumPartitions())

orders_coalesced = orders_repartitioned.coalesce(2)
print("After coalesce:", orders_coalesced.rdd.getNumPartitions())

```

Trigger an action on each DataFrame and compare the Spark UI.

Answer:

- What does increasing the partition count change?
- When might too few partitions limit parallelism?
- When might too many partitions create overhead?
- Why does `repartition()` generally involve a shuffle?
- Why can `coalesce()` reduce partitions with less data movement?
- What trade-off can arise if you coalesce too aggressively?

Do not assume the same task count across every stage. Relate the task count to the partitions processed by that stage.

8.4 Write Parquet

```

monthly_summary.write.mode("overwrite").parquet(
    "output/monthly_sales_summary"
)

```

Inspect the output directory.

Explain:

- What is Parquet?
- Why is it commonly used for analytical data?
- Why can a write create multiple files?
- How are output tasks and partitions related to the files written?
- Why can producing too many tiny files become a problem?
- How could you control the number of output partitions before writing?

The number of files is not guaranteed to equal the DataFrame's partition count in every write scenario. Explain the typical relationship and its limitations.

9. Part 4 — Answer the business questions in Spark

Use the same business definitions as Assignment 3 wherever they apply. If you make a different decision, document it and explain the impact. Do not compare runtime as the main objective; compare correctness and understand the execution plan.

9.1 Finance — item sales by category over time

"How is item sales value changing across product categories over time?"

Build a DataFrame with these conceptual columns:

```
reporting_month | product_category | total_item_sales | distinct_orders
```

Suggested steps:

1. Join orders to order items using `order_id`.
2. Join products using `product_id`.
3. Join the category translation table using the category key.
4. Filter to delivered orders.
5. Parse the purchase timestamp and derive `reporting_month`.
6. Group by reporting month and English product category.
7. Calculate `SUM(price)` and `COUNT(DISTINCT order_id)`.
8. Order the output by month and category.

Metric definition: price is item price. Exclude `freight_value` from `total_item_sales`. Do not label this metric as total company revenue.

Join design decision: Check whether all product category keys have a translation. Decide whether the translation join should be inner or left. Explain what happens to untranslated categories under each choice. Preserve unmatched categories if your reporting requirement says no product sales should disappear.

Inspect the plan:

```
finance_df.explain(True)
```

Identify the parsed, analysed and optimised logical plans, and the physical plan. Look for join strategies and Exchange / shuffle steps. Remember: `explain()` inspects the plan; it does not execute the query.

Trigger an action, open the SQL tab, and inspect the query details.

Validation against Assignment 3:

- Does total item sales across all months match?
- Does the number of reporting-month/category combinations match?
- Are category names and null/untranslated categories handled consistently?
- If results differ, trace the difference to filters, join types, timestamp parsing, duplicate handling or metric definitions.

9.2 Operations — delivery performance by customer state

"How is delivery performance changing, and where are late deliveries occurring?"

Build a DataFrame with:

```
reporting_month | customer_state | delivered_orders | late_orders | late_rate
```

Suggested steps:

1. Join orders to customers using `customer_id`.
2. Parse purchase, actual delivery and estimated delivery dates as timestamps.
3. Derive a delivery status:
 - `not_delivered` when the actual delivery date is null.
 - `late` when actual delivery is after estimated delivery.
 - `on_time` when actual delivery is on or before estimated delivery.
4. Derive reporting month from purchase timestamp.
5. Group by reporting month and customer state.
6. Calculate delivered orders, late orders and late rate.

Use an explicit null-handling rule. Do not silently classify an undelivered order as an on-time delivery.

Business definition for this assignment:

$$\text{late_rate} = \text{late delivered orders} / \text{all delivered orders}$$

Undelivered orders are excluded from both the numerator and denominator of this rate. You may report them separately if useful, but do not mix them into delivered-order performance.

Questions:

- Why is an undelivered order different from an on-time delivery?
- What should happen if either delivery date is null?
- What happens if the estimated date is null?
- Why should the denominator be delivered orders rather than all orders?
- What should happen to a month/state group with zero delivered orders?

Validate the total late delivered-order count and at least one state's monthly figures against Assignment 3. Make sure the definitions are identical before comparing.

10. Spark UI — mandatory observations log

Maintain this table in your notebook. Fill it using your actual observations; do not invent or copy expected numbers.

Step	Action	Jobs	Stages	Tasks	Shuffle / Exchange?	Cache?	Observation
Count orders	count()						
Delivered filter	count()						
Join orders/items	count()						
Monthly aggregation	show()						
Deduplication	count()						
Repartition	count()						
Coalesce	count()						
Finance output	show()						
Operations output	show()						
Parquet write	write()						

For at least three representative executions, record:

- Input size or records read, where shown.
- Shuffle read and shuffle write, where shown.
- A task-duration observation (for example, whether task durations are similar or highly uneven).
- What you think explains the observed behaviour.

Screenshots required:

1. Jobs or Stages page showing a multi-stage execution.
2. SQL tab showing a query plan / execution graph.
3. Storage tab showing the cached Finance base.

Embed screenshots in the notebook and add a short caption explaining what each screenshot demonstrates. Spark UI metrics vary with local environment, Spark version, file layout and configuration. Your explanation matters more than matching somebody else's numbers.

11. Engineering reflection

Answer in your own words:

1. **Lazy evaluation:** Give one example where Spark combined multiple transformations before execution. What benefit does that provide?
2. **Narrow vs wide:** Compare `filter()` and `groupBy()`. Why do they have different implications for execution?
3. **DAG and stages:** Explain how a shuffle boundary affects stage creation.
4. **Partitions and tasks:** Why is a task count not a fixed property of a `DataFrame`?
5. **Caching:** Which `DataFrame` did you cache, where did you reuse it, and what did the Storage tab show? When would caching be wasteful?

6. **Join strategy:** What join strategy did Spark choose for one of your joins? What factors can influence that choice?
7. **Spark vs PostgreSQL:** For this Olist dataset, why might PostgreSQL be simpler or faster? What workload characteristics might make Spark worth the additional complexity?
8. **Spark UI:** If a job is slow, which metrics or tabs would you inspect first, and what would you look for?
9. **Data quality:** Why should a pipeline define its grain and duplicate-resolution rule before aggregating?
10. **Business definitions:** How did you prevent undelivered orders from being counted as on-time deliveries?

12. Deliverables and submission checklist

Submit one notebook:

```
notebooks/assignment_4_spark.ipynb
```

It must contain:

- A working SparkSession and environment/version notes.
- All required Olist DataFrames and schema observations.
- Transformations with prediction → execution → observation.
- Narrow/wide transformation explanation.
- Join-plan and shuffle investigation.
- Deduplication check and controlled duplicate experiment if needed.
- Actions and Parquet output exercise.
- Cache experiment using a DataFrame actually reused in Finance.
- Repartition and coalesce comparison.
- Finance and Operations outputs with validation against Assignment 3.
- Spark UI observations log and at least three embedded screenshots.
- Answers to all ten reflection questions.

Checklist

- ☐ I can explain transformations, actions and lazy evaluation.
- ☐ I can distinguish narrow and wide transformations.
- ☐ I can explain partitions, tasks, stages, jobs and shuffle.
- ☐ I inspected a physical plan using `explain(True)`.
- ☐ I compared a join strategy and understand broadcast joins.
- ☐ I tested caching and released the cache with `unpersist()`.
- ☐ I compared `repartition()` and `coalesce()`.
- ☐ I wrote a Parquet output and inspected the files.
- ☐ Finance results match Assignment 3 under the same metric definitions.
- ☐ Operations results use a clearly defined delivered-order denominator.
- ☐ I included the Spark UI screenshots and observations.
- ☐ I committed meaningful changes without committing generated output or source data unnecessarily.

13. Git checkpoints

Use meaningful commits. Adapt the messages to the work you actually completed.

```
git add notebooks/assignment_4_spark.ipynb requirements.txt .gitignore
git commit -m "Set up Spark and inspect Olist schemas"

git add notebooks/assignment_4_spark.ipynb
git commit -m "Explore Spark transformations and execution plans"

git add notebooks/assignment_4_spark.ipynb
git commit -m "Investigate actions partitions caching and Parquet"

git add notebooks/assignment_4_spark.ipynb
git commit -m "Build Finance and Operations Spark outputs"

git add notebooks/assignment_4_spark.ipynb
git commit -m "Document Spark UI observations and reflections"
```

Do not commit large source CSVs, generated Parquet output, temporary Spark files or notebook checkpoint folders unless your repository instructions explicitly require them. Add suitable generated paths to `.gitignore`.

14. Keep the scope focused

Do not add these topics to this assignment:

- Structured Streaming.
- Spark MLlib.
- Delta Lake or Iceberg.
- Kubernetes, YARN or cluster setup.
- Cloud storage integration.
- Airflow orchestration.

Those topics can come later.

Local Spark uses the same fundamental concepts of DataFrames, plans, partitions, tasks, stages and shuffles. However, a local run does not reproduce all the resource, network, fault-tolerance and operational characteristics of a multi-machine cluster.

15. A final word from your team lead

Imagine this pipeline is running on a ten-node Spark cluster next month.

Your team lead asks:

- “Why is this stage taking 40 minutes?”
- “Why are some tasks taking much longer than others?”
- “Where is the shuffle happening?”
- “Are we moving more data than we need to?”
- “Are we caching something that is never reused?”

A capable data engineer does not just know which function to call. They can explain what the engine is doing, interpret the evidence, and make a reasoned engineering decision.

Don't focus only on getting the answer. Focus on understanding how the engine gets there.

— Uma Kiran | DataKern | www.data-kern.com